Mathematical Description of `KernelDensityEstimator`

Contents

1 Epanechnikov Kernel **1**

2 KDE Estimate **2**

3 Point Events: Sliding-Window Accumulator **2**

4 Interval Events: Step Decomposition **2**
4.1 Three-Phase Lifecycle 3
4.2 Transitioning Accumulator Expansion 3

5 Boundary Correction **3**

6 Confidence Intervals **4**
6.1 Effective Observation Time 4
6.2 Effective Count and Garwood Interval 4

7 Summary of Complexity **4**

This document describes the mathematics underlying each method of the `KernelDensityEstimator` Kotlin class. The estimator converts a stream of Geiger–Müller counts into a continuous dose-rate curve $\hat{\lambda}(t)$ in μSv/h, together with Garwood Poisson confidence intervals.

1 Epanechnikov Kernel

The class uses the *Epanechnikov* kernel with bandwidth $h > 0$:

$$K_h(s) = \frac{3}{4h}\left(1 - \frac{s^2}{h^2}\right)\mathbf{1}_{\{|s|\le h\}}. \tag{1}$$

It is a symmetric, non-negative, compactly supported kernel that integrates to one and minimises asymptotic mean integrated squared error among all kernels.

Kernel CDF. The antiderivative of K_h, needed for interval convolution (Section 4), is

$$F_K(x) = \int_{-\infty}^{x} K_h(s)\,ds = \begin{cases} 0 & x < -h, \\ \dfrac{3}{4}\left(\dfrac{x}{h} - \dfrac{x^3}{3h^3} + \dfrac{2}{3}\right) & |x| \le h, \\ 1 & x > h. \end{cases} \tag{2}$$

Helper `epanechnikovIntegral`(u_ℓ, u_r)**.** Returns the partial mass of the kernel over a normalised sub-interval:

$$\texttt{epanechnikovIntegral}(u_\ell, u_r) = \int_{u_\ell}^{u_r} \tfrac{3}{4}(1-u^2)\,du = \tfrac{3}{4}\Big[u - \tfrac{u^3}{3}\Big]_{u_\ell}^{u_r}. \tag{3}$$

On full support $[-1, 1]$ this equals 1. It is used for boundary correction (Section 5).

Helper `epanechnikovSquaredIntegral`(u_ℓ, u_r)**.** Returns the squared-kernel integral needed for T_{eff}:

$$\texttt{epanechnikovSquaredIntegral}(u_\ell, u_r) = \int_{u_\ell}^{u_r} \left(\tfrac{3}{4}\right)^2 (1-u^2)^2 du = \tfrac{9}{16}\Big[u - \tfrac{2u^3}{3} + \tfrac{u^5}{5}\Big]_{u_\ell}^{u_r}. \tag{4}$$

On full support $[-1, 1]$ this equals 3/5.

2 KDE Estimate

Let the data consist of N point events at times t_i with counts n_i, and independently of L uniformly distributed sample intervals $[a_\ell, b_\ell]$ with counts c_ℓ (Section 4). Let t_{start} and t_{end} be the overall data extents, covering both point events and interval endpoints.

The boundary-corrected KDE estimate at query time t is

$$\hat{\lambda}(t) = \frac{\displaystyle\sum_{i=1}^{N} n_i\, K_h(t - t_i) \;+\; \Lambda_{\text{iv}}(t)}{k_m(t)}, \tag{5}$$

where $\Lambda_{\text{iv}}(t)$ is the interval contribution (Section 4) and $k_m(t)$ is the boundary-correction mass (Section 5). The estimate is in cps; multiplication by the calibration factor α [μSv/h/cps] gives dose rate.

3 Point Events: Sliding-Window Accumulator

Implemented in `estimateDoseRateHelper`*.*

For a query t, only events with $|t - t_i| \le h$ contribute. Introducing the numerical-stability offset $\tau = t - t_{\text{start}}$ and $\tau_i = t_i - t_{\text{start}}$, the point-event numerator of (5) is

$$\begin{aligned}\sum_{|t-t_i|\le h} n_i K_h(t-t_i) &= \frac{3}{4h} \sum_{|t-t_i|\le h} n_i \left(1 - \frac{(\tau-\tau_i)^2}{h^2}\right) \\ &= \frac{3}{4h}\left(S_0 - \frac{\tau^2 S_0 - 2\tau S_1 + S_2}{h^2}\right),\end{aligned} \tag{6}$$

where $S_k = \sum_{|t-t_i|\le h} n_i\, \tau_i^k$ for $k = 0, 1, 2$.

As t moves rightward over the sorted query array, the window $[t - h, t + h]$ slides, and S_0, S_1, S_2 are updated in $O(1)$ per admitted or evicted event. Total complexity is $O(N + Q)$ for Q query points.

4 Interval Events: Step Decomposition

Implemented in `addSampleInterval` *and* `estimateDoseRateHelper`*.*

An interval $[a, b]$ with c counts models a uniform count-rate density $w = c/(b-a)$ [cps]. Its exact contribution to the KDE numerator is

$$\Lambda_{[a,b]}(t) = w \int_a^b K_h(t-\tau)\, d\tau = w\big[F_K(t-a) - F_K(t-b)\big]. \tag{7}$$

This expresses the interval as two *step events*: a rising step at a with weight $+w$ and a falling step at b with weight $-w$:

$$\Lambda_{[a,b]}(t) = (+w)\, F_K(t-a) + (-w)\, F_K(t-b). \tag{8}$$

For M step events at times $\{t_j\}$ with weights $\{w_j\}$ (so $2L = M$), the total interval contribution is

$$\Lambda_{\text{iv}}(t) = \sum_{j=1}^{M} w_j\, F_K(t-t_j). \tag{9}$$

4.1 Three-Phase Lifecycle

As t sweeps rightward, step event j passes through three phases:

Phase	Condition	Contribution
Inactive	$t < t_j - h$	0
Transitioning	$\lvert t - t_j\rvert \le h$	$w_j F_K(t-t_j)$ (cubic in t)
Completed	$t > t_j + h$	$w_j \cdot 1$ (constant)

The *completed* events contribute only their weights summed into a scalar Σ_{done}, maintained by adding w_j when event j exits the transitioning window.

4.2 Transitioning Accumulator Expansion

For transitioning events, introduce $\tau_j = t_j - t_{\text{start}}$ and $\tau = t - t_{\text{start}}$. Substituting (2) and expanding $(t-t_j)^3 = (\tau-\tau_j)^3$:

$$w_j\, F_K(t-t_j) = \frac{3}{4}\, w_j \left(\frac{\tau-\tau_j}{h} - \frac{(\tau-\tau_j)^3}{3h^3} + \frac{2}{3} \right). \tag{10}$$

Collecting by powers of τ_j and defining moment accumulators

$$\tilde{T}_k = \sum_{\text{transitioning}\, j} w_j\, \tau_j^k, \quad k = 0, 1, 2, 3, \tag{11}$$

the sum over all transitioning events evaluates to

$$\Lambda_{\text{trans}}(\tau) = \frac{3}{4}\left[\tilde{T}_0 \left(\frac{\tau}{h} - \frac{\tau^3}{3h^3} + \frac{2}{3} \right) + \tilde{T}_1 \left(\frac{\tau^2}{h^3} - \frac{1}{h} \right) - \tilde{T}_2\, \frac{\tau}{h^3} + \tilde{T}_3\, \frac{1}{3h^3} \right]. \tag{12}$$

The accumulators $\tilde{T}_0, \ldots, \tilde{T}_3$ are updated in $O(1)$ per admitted or evicted step event, exactly mirroring the S_0, S_1, S_2 treatment of point events. The total interval contribution is then

$$\Lambda_{\text{iv}}(t) = \Sigma_{\text{done}} + \Lambda_{\text{trans}}(\tau). \tag{13}$$

The sliding window for step events admits event j when $t_j - h \le t$ and evicts it (to Σ_{done}) when $t_j + h \le t$. Admission is processed before eviction so that a step event whose entire kernel support has already passed is immediately routed to Σ_{done} without corrupting the $\tilde{T}_k$ accumulators.

Overall complexity with L intervals (i.e. $M = 2L$ step events) is $O(N + M + Q)$.

5 Boundary Correction

Implemented in `estimateDoseRateHelper` and `getConfidenceIntervals`.

Near the endpoints of the data, the kernel window $[t-h, t+h]$ extends beyond the data range $[t_{\text{start}}, t_{\text{end}}]$. The *renormalisation* (boundary) correction divides the KDE numerator by the fraction of the kernel that lies within the data range.

Using the substitution $u = (t-\tau)/h$ (so $\tau = t_{\text{start}}$ maps to $u_r = (t - t_{\text{start}})/h$ and $\tau = t_{\text{end}}$ maps to $u_\ell = (t - t_{\text{end}})/h$), the boundary-correction kernel mass is

$$k_m(t) = \int_{t_{\text{start}}}^{t_{\text{end}}} K_h(t-\tau)\, d\tau = \texttt{epanechnikovIntegral}(u_\ell,\, u_r), \tag{14}$$

where

$$u_\ell = \max\left(-1,\ \frac{t - t_{\text{end}}}{h}\right), \qquad u_r = \min\left(+1,\ \frac{t - t_{\text{start}}}{h}\right). \tag{15}$$

For t in the interior (both kernel edges inside the data range), $u_\ell = -1$, $u_r = +1$, and $k_m = 1$.

It is essential that t_{start} and t_{end} span *interval endpoints* as well as point-event timestamps; otherwise the correction overestimates the boundary effect at the edges of long intervals.

6 Confidence Intervals

Implemented in `getConfidenceIntervals`.

6.1 Effective Observation Time

Locally, the KDE behaves as if n_{eff} counts were observed over an equivalent observation window of duration T_{eff}. By matching the asymptotic Poisson variance of the KDE to that of a direct observation,

$$T_{\text{eff}} = \frac{k_m(t)^2}{\displaystyle\int_{t_{\text{start}}}^{t_{\text{end}}} K_h^2(t-\tau)\, d\tau}. \tag{16}$$

The denominator is computed by substituting $u = (t-\tau)/h$, giving $d\tau = -h\, du$:

$$\int_{t_{\text{start}}}^{t_{\text{end}}} K_h^2(t-\tau)\, d\tau = \frac{1}{h}\ \texttt{epanechnikovSquaredIntegral}(u_\ell, u_r), \tag{17}$$

so that

$$T_{\text{eff}} = \frac{h\ k_m(t)^2}{\texttt{epanechnikovSquaredIntegral}(u_\ell, u_r)}. \tag{18}$$

For an interior point with full kernel support, this reduces to $T_{\text{eff}} = h \cdot 1^2/(3/5) = 5h/3$.

6.2 Effective Count and Garwood Interval

The effective count is

$$n_{\text{eff}} = \hat{\lambda}(t)\ T_{\text{eff}}, \tag{19}$$

where $\hat{\lambda}(t)$ is the cps estimate from (5).

The *Garwood* exact Poisson confidence interval at confidence level $1-\alpha$ for an observed integer count n is

$$\left(\frac{\chi^2_{2n,\,\alpha/2}}{2},\quad \frac{\chi^2_{2(n+1),\,1-\alpha/2}}{2}\right), \tag{20}$$

expressed here as bounds on the true count. Dividing by T_{eff} and multiplying by the calibration factor α converts to μSv/h.

Because n_{eff} is in general non-integer, the bounds are linearly interpolated between the two bracketing integers $n_F = \lfloor n_{\text{eff}} \rfloor$ and $n_C = n_F + 1$ using weight $\xi = n_{\text{eff}} - n_F$:

$$\chi_L = \chi^2_{2n_F,\,0.025} + \xi\big(\chi^2_{2n_C,\,0.025} - \chi^2_{2n_F,\,0.025}\big)\,, \tag{21}$$

$$\chi_R = \chi^2_{2(n_F+1),\,0.975} + \xi\Big(\chi^2_{2(n_C+1),\,0.975} - \chi^2_{2(n_F+1),\,0.975}\Big)\,. \tag{22}$$

The dose-rate confidence bounds are

$$\lambda^{\pm}\,[\mu\text{Sv/h}] = \frac{\chi_{L,R}}{2\,T_{\text{eff}}} \cdot \alpha. \tag{23}$$

7 Summary of Complexity

Operation	Per-call cost	Amortised total
`addPoint`	$O(1)$	$O(N)$
`addSampleInterval`	$O(1)$	$O(L)$
`estimateDoseRateHelper`	$O(N + M + Q)$	—
`getConfidenceIntervals`	$O(N + M + Q)$	—

N = number of point events, L = number of intervals, $M = 2L$ = number of step events, Q = number of query points.